



AEGIS-RL: Abstract, Explainable Graphs for Integrated Safety in RL

Tuan Le¹ · Risal Shefin² · Debashis Gupta² · Thai Le³ · Sarra Alqahtani²

Received: 2 September 2025 / Accepted: 16 April 2026
© The Author(s) 2026

Abstract

Ensuring the safety of reinforcement learning (RL) policies in high-stakes environments requires more than formal verification: it needs interpretability and targeted falsification—the deliberate search for counter-examples that expose potential failures before deployment. We present AEGIS-RL (Abstract, Explainable Graphs for Integrated Safety in RL), a hybrid framework that unifies (1) explainable RL, (2) probabilistic model checking, and (3) risk-guided falsification, and augments them with (4) a lightweight runtime safety shield that switches to a fallback policy when estimated risk exceeds a threshold. AEGIS-RL first builds a directed, semantically meaningful graph from offline trajectories that blends local and global explanations to make policy behavior transparent and verifier-friendly. This abstract graph is fed to a probabilistic model checker (e.g., Storm) to verify temporal safety specifications; when violations exist, the checker returns interpretable counterexample traces that pinpoint how the policy fails. When specifications appear satisfied, AEGIS-RL estimates residual risk during checking to steer falsification toward high-risk, under-explored states, broadening coverage beyond the offline data. Across safety-critical benchmarks including two MuJoCo tasks and a medical insulin-dosing scenario; AEGIS-RL uncovers significantly more violations than uncertainty- and fuzzing-based baselines and yields a broader, more novel set of failure trajectories. The resulting explanations and counterexamples provide actionable guidance to understand, debug, and repair unsafe policies while enabling runtime mitigation without retraining.

Keywords Explainable reinforcement learning · Formal verification · Risk-guided falsification · Policy interpretability · Safety assurance

1 Introduction

Reinforcement Learning (RL) has achieved remarkable performance in games and simulated environments. However, deploying RL in real-world, especially in safety-critical domains such as autonomous driving, robotics, and healthcare remains a challenge. This gap is primarily due to two closely interrelated challenges: (1) ensuring the *safety* of RL policies and (2) providing *interpretation* for their decisions. While safe RL and explainable

Extended author information available on the last page of the article

RL (XRL) have been studied extensively, they have been rarely integrated in a way that directly enhances safety assurance.

Research in *safe RL* has traditionally focused on training agents that adhere to explicit safety constraints [1–6]. Meanwhile, *explainable RL* (XRL) research emphasizes on improving the transparency and enable human interpretability of RL agents' behaviors [7]. While the two research directions are often pursued separately, they are inherently connected: explicit safety constraints (e.g., avoid obstacles and keep the gas up to a minimum threshold) alone cannot cover all possible situations an RL agent may encounter, and failures are inevitable in deployment. In such cases, explanations provided by XRL (e.g., the agent hit the obstacles to reach to the gas station before reaching the minimum threshold) are not merely a usability feature but a critical component of safety assurance. In safety-critical deployments, safe RL and explainable RL reinforce each other in two directions: (i) safety mechanisms are only practically useful when practitioners can understand *why* a policy behaved safely or unsafely in a specific situation, and (ii) explanations are safety-relevant when they reveal which incentives, assumptions, or generalization errors led the policy toward hazardous outcomes and how to prevent recurrence. We illustrate this connection with two scenarios. Consider an RL policy controlling a vehicle with the safety requirement “reach the destination without collision.” Many real failures are not caused by a single obviously wrong action, but by a *sequence* of locally reasonable decisions that gradually increase risk (e.g., choosing a narrow passage to save time, then accelerating to recover from delay, then failing to yield when visibility is limited). In such cases, constraint satisfaction during training does not automatically translate to reliable safety at deployment, because rare disturbances (sensor noise, unexpected obstacles, distribution shift) can push the policy into regimes it has not experienced. Explainability contributes to safety by revealing which decision factors dominated (e.g., urgency to reach the goal versus caution near obstacles), whether the policy was acting on spurious cues, and where the policy's behavior becomes brittle. Conversely, safety analysis contributes to explainability by focusing attention on the most consequential parts of behavior: it identifies the critical decision points and trajectories that lead to near-misses or collisions, enabling developers to diagnose root causes rather than relying on aggregate performance metrics. Consider, in another example that we study in this paper, an RL policy that recommends insulin doses based on physiological signals, where the safety objective is to avoid hypoglycemia while maintaining glucose control. Here, unsafe outcomes can be delayed and context-dependent: a dose that appears appropriate under typical meal absorption can become dangerous after exercise, illness, or other unmodeled variability. This makes safety inseparable from interpretability, because clinicians and developers must understand the rationale behind a recommendation, how sensitive it is to context, and which patterns of decisions precede unsafe events. Explainability supports safety by surfacing why a particular dose was chosen and how the policy balances short-term glucose reduction against longer-horizon hypoglycemia risk; safety-focused evaluation, in turn, makes explanations actionable by highlighting the rare, high-impact contexts where misunderstanding or misgeneralization is most harmful. In both settings, the key challenge is not only detecting whether unsafe behavior exists, but producing explanations that localize *how* and *under what conditions* unsafe behavior emerges.

Specifically, in this work, we hypothesize that clear, faithful explanations do not only allow practitioners to assess whether agents are adhering to constraints, identify the root

causes of violations when they occur, but also empower end-users to take timely intervention before harm propagates.

In other words, this work advances the position that *safety verification and interpretability should not be treated in isolation but must be tightly integrated*

Nevertheless, formally verifying the safety of RL policies remains challenging. First, RL operates in continuous, high-dimensional state spaces, causing state explosion; practical verification therefore relies on constructing finite, verifier-friendly *abstractions* (e.g., predicate- or graph-based abstract MDPs), but building complete abstractions is computationally infeasible [8]. Second, unlike traditional settings with known dynamics, RL policies are learned black boxes, which necessitates *data-driven abstractions* extracted from trajectories to obtain interpretable, checkable models [9]. Third, training uses finite datasets or limited rollouts, leaving coverage gaps; as a result, verification on an abstraction may overlook underexplored regions and introduce blind spots [7]. Fourth, when violations occur, practitioners need human-readable counterexamples and causal summaries to localize and repair faults, yet interpretability in RL and AI in general is often treated as a usability add-on rather than a development tool. Finally, many existing safety workflows are *reactive* (i.e., they respond after a failure is observed), frequently resorting to broad retraining or reward redesign to fix a localized issue [3, 4]; a costly remedy when targeted falsification and lightweight runtime safeguards could mitigate the problem without retraining.

These limitations underscore that *formal verification alone is insufficient*. Because abstracted models inevitably omit details of the underlying policy, verification can overlook subtle vulnerabilities or untested corner cases. Thus, safety falsification; i.e., or deliberate search for counterexamples that expose policy violations of safety specifications [10, 11], can be used as a complementary method to find these blind spots. To the best of our knowledge, no prior work has unified falsification, interpretability, and formal verification into a single approach that simultaneously addresses these complex challenges.

Therefore, in this work, we first propose a two separate XRL methods to provide local and global explanations and integrate them into a directed graph that serves as an interpretable abstraction of RL policies. Secondly, we develop a verification approach that combines formal model checking with risk- and uncertainty-guided falsification. The directed graph explains safety violations by showing how task requirements or safety constraints influence decisions. We apply a local explanation method that train two critics, Q_{task} and Q_{risk} , to estimate returns and risks for a state s_t , action a_t , and policy π [12]. These post-hoc Q-functions can be applied to any fixed policy without accessing its internal structure. For global explanations, we extend our previous work, CAPS graph [13], enhancing nodes (abstract states) with Q_{task} and Q_{risk} , and annotating edges (actions) with motivations rooted in task or safety requirements. This graph serves as an input to the formal verification process, which verifies temporal logic specifications over the policy abstraction using model checking. If a violation is detected, we conclude that the policy is unsafe. However, if verification passes, safety cannot be guaranteed due to potential abstraction omissions or dataset limitations. To mitigate this, we use risk estimates from the formal verification to guide a falsification strategy focused on high-risk and underrepresented regions of the state space. Specifically, we combine risk critics and epistemic uncertainty to identify violations that formal verification might miss.

Contributions Our main contributions include:

1. **Local and Global Explanation Graph:** A unified representation that combines local and global explanations to highlight both safety violations and compliance, expressed as a directed graph. This graph provides users with both trust and practical utility for understanding and diagnosing RL agents in safety-critical scenarios.
2. **Explanation-Based Formal Verification:** We use the explanation graph as an abstraction input to the model checker for safety constraint verification. This enables formal safety guarantees with transparent counterexamples.
3. **Risk- and Uncertainty-Guided Falsification:** To deal with model checking missing safety violations due to abstraction, we develop a targeted falsification strategy guided by risk estimates and ensemble-based uncertainty, focusing on high-risk and poorly represented states.
4. **Runtime Policy Repair:** We introduce a lightweight shielding mechanism that switches to a fallback policy in detected high-risk states, enabling modular policy repair without retraining.

To evaluate the interpretability of our explanation graph, we conduct a user study measuring users' comprehension of vulnerable and safe policies under adversarial conditions, across 9 distinct scenarios. We then evaluate our formal verification, falsification, and policy repair methods extensively across benchmark domains, including CMDP Mujoco Navigation2, Maze, and a simulated medical task for regulating insulin dosing in type 2 diabetes patients.

Empirical results indicate that our local-global explanation graph significantly enhances users' understanding of policy behavior. Furthermore, our verification and falsification approach outperforms baseline methods, detecting more safety violations while maintaining transparency and enabling modular interventions. By unifying explainable policy abstraction, formal verification, and guided falsification, this work advances the state of the art in RL safety assurance and lays a practical foundation for deploying RL in high-stakes environments.

2 Background

Related work XRL methods have evolved to offer critical insights into the behavior of RL agents, typically categorized into either local or global explanations [14]. Global explanation techniques aim to summarize an agent's policy across the entire state space [13, 15–18], while local explanations focus on justifying individual decisions at specific time steps, often through post-hoc methods such as saliency maps [19, 20] or reward decomposition [21]. Although some hybrid approaches attempt to bridge the gap by overlaying local saliency-based justifications onto global strategy summaries [19], they generally fall short in capturing safety-critical aspects of agent behavior, such as estimating failure risk at each state or across the entire policy. Notably, existing XRL approaches lack the ability to explicitly explain how and why a policy satisfies or violates formal safety constraints. Overall, none of existing XRL methods provide explicit explain of how policies satisfy or violate formal safety constraints. Our local and global representation graph can solve this problem and convey the satisfaction or violation of safety constraints explicitly.

Safety assurance in RL has been approached through both formal verification and falsification techniques, each addressing complementary aspects of the problem. Formal verification in RL often leverages probabilistic model checking tools such as PRISM [22]

and Storm [23] that provide rigorous guarantees by verifying temporal logic properties (e.g., PCTL) over Markov decision processes. However, applying these tools to RL is challenging due to the large or continuous state spaces and the black-box nature of learned policies. To enable tractable verification, prior work has explored abstraction techniques, including state aggregation and symbolic representations [24]. However, they often sacrifice or do not consider model interpretability that is critical for understanding and mitigating safety violations.

Falsification methods offer a complementary strategy by actively searching for counterexamples that reveal unsafe behaviors. Such methods include random search, optimization-based falsification, and RL-guided falsifiers [10, 11]. While these techniques are scalable and can expose vulnerabilities in complex systems, they lack formal guarantees and often require significant computational effort. To bridge the gap between exhaustive verification and scalable falsification, some recent approaches combine model checking with runtime shielding [5] by using formal tools to detect violations and then switching policies dynamically to avoid them. Nonetheless, existing methods tend to treat abstraction, interpretability, and falsification as separate concerns. In contrast, our work builds on these foundations by tightly integrating explainable policy abstraction, model checking, and risk-aware falsification into a single framework. Particularly, we use CAPS to generate a semantically meaningful abstraction from offline trajectories, enabling formal verification through Storm. When model checking fails to detect violations such as due to limited coverage or abstraction, we use the risk estimates from the abstract model to guide a targeted falsification process. This combination provides formal safety analysis with practical falsification capabilities while supporting interpretability throughout.

Probabilistic computation-tree logic Probabilistic Computation Tree Logic (PCTL) is a temporal logic for specifying probabilistic safety and reachability properties in stochastic systems [25]. It extends CTL by enabling probability constraints over system behaviors, and is widely used in popular model checking tools like PRISM [25] and Storm [26]. PCTL formulas follow the grammar:

$$\begin{aligned}\Phi &::= a \mid \neg\Phi \mid \Phi \wedge \Phi \mid \mathbb{P}_J(\phi) \\ \phi &::= X\Phi \mid \Phi \ U \ \Phi \mid \Phi \ U^{\leq n} \Phi\end{aligned}$$

Here, a is an atomic proposition; $\neg$ and $\wedge$ are logical negation and conjunction. $\mathbb{P}_J(\phi)$ means the path formula ϕ holds with probability in interval $J \subseteq [0, 1]$. The main temporal operators are; $X\Phi$: Φ holds in the next state (Next), $\Phi_1 \ U \ \Phi_2$: Φ_1 holds until Φ_2 becomes true (Until), and $\Phi_1 \ U^{\leq n} \ \Phi_2$: same as above but within n steps (Bounded Until). For example, $\mathbb{P}_{\leq 0.01}(U^{\leq 100} \text{ unsafe})$ states that reaching an unsafe state within 100 steps has probability at most 0.01.

CAPS In this paper, we build on our previous method, CAPS [13, 27], a recently introduced global explanation XRL method that has been successful in providing comprehensible summaries of RL policies. CAPS collects natural language (NL) predicates from the user and gathers up to 500 timesteps from the RL agent’s trajectories. To simplify the explanation process, it uses the CLTree [28] clustering algorithm to abstract the agent’s states into clusters, forming a hierarchy. A heuristic optimization then selects the best cluster configuration based on state transition accuracy and user interpretability. CAPS constructs the agent’s policy (π

) as a directed graph $G = (V, E)$, where set of nodes V represent abstract states (clusters), and set of edges E show the agent's actions and transition probabilities. CAPS enriches the graph by labeling abstract states with English explanations based on user-defined predicates and boolean algebra. We chose CAPS because it offers a comprehensive global explanation, revealing the agent's policy across the state space rather than focusing on specific states. It also outperforms other global XRL methods [18, 29] in fidelity and user comprehension. While alternatives like ASAP [30] offer attention-based summarization, they lack the explicit semantic structure needed for model checking. Similarly, ATA [31] requires altering the training process, whereas CAPS allows us to abstract and verify pre-existing policies from offline data. While CAPS [13] can explain an RL agent's policy, it cannot address the impact of safety violations on the agent's behavior, nor does it enable users to debug specific safety concerns. CAPS lacks the ability to provide quantified risk assessments necessary for formal safety verification. Consequently, CAPS alone cannot explain the causal mechanisms behind why a policy satisfies or violates specific formal constraints. *This work addresses these limitations by providing safety through enhanced explainability and transparency.*

Positioning relative to our prior work [12, 13, 27, 32] CAPS [13, 27] provides a faithful and human-interpretable *global* abstraction of a policy by clustering states into semantically labeled nodes and summarizing action-conditioned transitions as a directed graph. However, CAPS is intentionally task-centric: it does not quantify safety risk, does not label unsafe regions, and therefore cannot support debugging of safety failures or verification of formal temporal safety constraints. Our subsequent work, xSRL [12], augments the CAPS abstraction with post-hoc task and risk estimates to improve safety-aware interpretation and enable policy patching; however, it does not provide a formal verification mechanism for temporal logic specifications, nor does it generate formal counterexample traces when specifications are violated. In parallel, AdvExRL [32] introduces a safe RL training procedure with a learned risk estimator, but it does not leverage that estimator to construct a semantically interpretable abstraction suitable for probabilistic model checking. In this submission, we close these gaps by using the CAPS-style semantic abstraction as a verifier-compatible surrogate and integrating quantified risk information into a workflow that supports checking temporal safety specifications and producing interpretable counterexamples, thereby elevating explainability from a diagnostic tool to a foundation for formal safety analysis.

3 Problem setting

We consider a standard Constrained Markov Decision Process (CMDP), $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mu, \mathcal{P}, \mathcal{R}, \gamma, \mathcal{C}, T)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, μ is the initial distribution, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the transition probability dynamics, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, γ is the discount factor, and $\mathcal{C} = \{c_i : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}\}_{i=1}^k$ specifies the costs for safety constraint violations in any trajectory $\tau = \{s_0, a_0, \dots, a_{T-1}, s_T\}$ over horizon T . CMDPs capture the dual objectives of maximizing reward while respecting safety constraints, making them well-suited for formal verification. We assume that reaching the task goal or violating a safety constraint terminates the episode. The objective of the agent's policy, we call it *task policy* hereafter, π_{task} is to learn the optimal control to maximize the expected discounted reward at time t ; $\mathcal{R}_{\pi_{task}} = \mathbb{E}_{\tau \sim \pi_{task}} [\sum_{t'=t}^T \gamma^{t'-t} r_{t'}]$. The task *policy* π_{task} is the solution to the CMDP.

We assume access to a fixed offline dataset $\mathbb{D} = \{X^{(i)}, r_i, c_i\}_{i=1:N}$, where $X^{(i)} = \{s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, c_t^{(i)}\}_{t=1:T}$ represents states, actions, rewards, and costs for episode i , our goal is to summarize these trajectories as a *graph* augmented with Q-value estimates for both the task reward and safety costs. We abstract π_{task} into a directed graph $\mathcal{G}_\pi = (V, E)$, where $V \subset \mathcal{S}$ are abstract states, $E \subseteq V \times V$ are transitions under π_{task} , and edges are annotated with empirical transition probabilities and safety costs. Let $f: \mathcal{S} \rightarrow V$ map each concrete state to an abstract node (cluster) in the policy graph. Given an offline dataset $\mathbb{D}$ generated by executing π_{task} , we define the empirical abstract transition probabilities by counting observed transitions between clusters:

$$\hat{P}(v' | v) = \frac{\sum_{(s_t, s_{t+1}) \in \mathbb{D}} 1\{f(s_t) = v, f(s_{t+1}) = v'\}}{\sum_{(s_t, s_{t+1}) \in \mathbb{D}} 1\{f(s_t) = v\}}.$$

We similarly define empirical per-node (or per-edge) cost summaries from the same trajectories. An abstract node v is labeled *unsafe* when its estimated risk exceeds a threshold, e.g., $\text{unsafe}(v) = 1\{\max_a Q_{\text{risk}}(v, a) > \epsilon\}$, which enables specifying temporal safety properties over $\mathcal{G}_\pi$.

Proposition 1 (*Finite-data reliability of abstract transition estimates*) Fix an abstract node $v \in V$ and successor $v' \in V$. Suppose we observe $n(v)$ transitions with $f(s_t) = v$ in the dataset, and let $p(v' | v) = \mathbb{P}(f(s_{t+1}) = v' | f(s_t) = v, \pi_{\text{task}})$ denote the true (policy-induced) abstract transition probability. Then, for any $\delta \in (0, 1)$, with probability at least $1 - \delta$,

$$|\hat{P}(v' | v) - p(v' | v)| \leq \sqrt{\frac{\log(2/\delta)}{2 n(v)}}.$$

This bound quantifies how limited coverage in $\mathbb{D}$ affects the fidelity of the abstract dynamics used for temporal safety analysis.

We then frame safety verification as probabilistic model checking on $\mathcal{G}_\pi$ against a temporal safety property φ (e.g., $\mathbb{P}_{\geq 0.95}[\neg \text{unsafe } \mathcal{U}^{\leq T} \text{ goal}]$). Abstraction-based verification, however, faces two main challenges: (1) **Incomplete Abstraction**—limited samples in $\mathcal{D}$ can omit critical transitions, producing incomplete or misleading graphs; and (2) **Epistemic Uncertainty**—sparse data yields unreliable transition and cost estimates in underexplored regions. Unlike uncertainty estimation in supervised active learning [33–35], where uncertainty measures prediction error over a fixed input distribution, CMDP uncertainty reflects incomplete knowledge of environment dynamics and safety outcomes. Offline RL [36, 37] similarly learns from fixed datasets but often lacks mechanisms for safety-aware reasoning or formal falsification. To address this, we introduce a risk-aware epistemic uncertainty metric that jointly captures insufficient coverage and potential safety violations. For instance, in Type 1 Diabetes management, π_{task} prescribes insulin doses from glucose and meal data; standard active learning may target high-variance glucose states, while offline RL may optimize glycemic control. Our approach instead prioritizes states with both high epistemic uncertainty and elevated hypoglycemia risk—such as post-exercise meals absent from $\mathcal{D}$ —enabling targeted exploration or falsification to address critical safety blind spots.

4 Approach

This section presents our approach that combines local and global explanations to enhance safety interpretation and combines formal verification with guided simulation-based search (Fig. 1). We (1) combine the local and global explanation into a directed graph to abstract the RL policy, (2) verify its safety properties using model checking via Storm. If no violations are found, we (3) perform targeted rollouts from high-risk and uncertain states to uncover violations missed by abstraction, and then (4) repair the policy utilizing the detected violating trajectories. Unlike standalone model checking, our method addresses abstraction gaps through targeted exploration; unlike heuristic search methods, it is formally guided by probabilistic risk estimates derived from model checking.

4.1 Safety interpretation via integrating local and global explanations

Local Explanation Method. Our local explanation method consists of 2 components: task reward estimation and risk estimation at each (s, a) pair. To estimate the future risk probability of a safety-constrained task policy π_{task} , we train a separate risk-critic Q_{risk} that evaluates the safety of a given state-action pair independently of task objectives. This risk-critic function $Q_{\text{risk}}^{\pi_{\text{task}}}(s, a)$, learned via:

$$Q_{\text{risk}}^{\pi_{\text{task}}}(s, a) = \mathbb{E}_{a_t \sim \pi_{\text{task}}(\cdot|s_t)} \left[\sum_{t'=t}^T \gamma^{t'-t} c(s_t, a_t) \right], \quad (1)$$

provides a way to assess the safety constraints of actions taken by the task policy. Here, $c(s_t, a_t)$ denotes the cost associated with violating safety constraints when taking action a_t in state s_t . In practice, we approximate $\hat{Q}_{\phi, \text{risk}}^{\pi_{\text{task}}}$, parameterized by ϕ , using sampled transitions (s_t, a_t, s_{t+1}, c_t) . This is done by minimizing the following MSE loss with respect to the target (RHS of (1)):

$$J_{\text{risk}}(s_t, a_t, s_{t+1}; \phi) = \frac{1}{2} \left(\hat{Q}_{\phi, \text{risk}}^{\pi_{\text{task}}}(s_t, a_t) - \mathbb{E}_{a_{t+1} \sim \pi(\cdot|s_{t+1})} [\hat{Q}_{\phi, \text{risk}}^{\pi_{\text{task}}}(s_{t+1}, a_{t+1})] \right)^2 \quad (2)$$

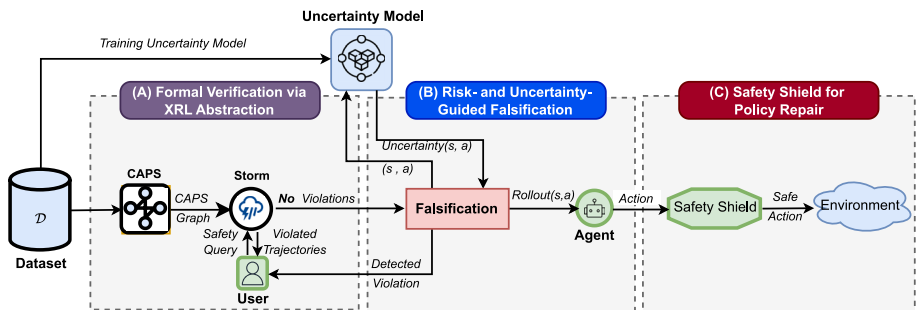


Fig. 1 Our hybrid approach combines (A) formal verification, (B) risk-aware falsification, and (C) explainable shielding for RL policy safety

To estimate the future task reward for a task policy π_{task} at each state, we train a separate task-critic Q_{task} similar to Q_{risk} in (1) but for the task reward r instead of safety cost c :

$$Q_{\text{task}}^{\pi_{\text{task}}}(s, a) = \mathbb{E}_{a_t \sim \pi_{\text{task}}(\cdot | s_t)} \left[\sum_{t'=t}^T \gamma^{t'-t} r(s_{t'}, a_{t'}) \right] \quad (3)$$

Since value-based RL algorithms, in general, and actor-critic policy-based algorithms estimate Q-function for the task policy, we can directly utilize their Q_{task} to explain their performance at each state in terms of task requirements. Learning a reward-value function is not, by itself, a novel contribution of this work. Our use of separate Q_{task} and Q_{risk} is motivated by (i) model-agnostic post-hoc auditing of fixed policies from trajectories and (ii) explicit decomposition of task utility versus safety risk, which is not available from reward-only critics or from scalarized objectives that entangle reward and cost. When a value-based RL algorithm already provides a reliable task critic, it can be reused in place of training Q_{task} .

Global explanation method To enhance the effectiveness of local explanations, we propose integrating both Q_{task} and Q_{risk} into the global explanation method, CAPS [13, 27]. This approach allows users to understand how the agent balances task objectives with safety constraints while presenting its overall strategy across episodes. CAPS summarizes the agent's policy in a directed graph where nodes represent abstract states and edges represent actions along with their transition probabilities, as described in Section 2. We choose to present local and global explanations in a directed graph format because it effectively illustrates the relationships and dependencies among states, actions, and outcomes; their causal and safety relationships; and the progression from one state to another based on specific actions. However, using CAPS alone as a global explanation tool did not provide sufficient insight for users to determine why the agent chose a particular action at a specific state, as demonstrated in our user studies (Section 5.1.2). Our method improves CAPS graph by: (1) incorporating local Q_{task} and Q_{risk} to show task and risk estimation at each abstract state, and (2) explicitly indicating whether each action is driven by task or safety considerations. We hypothesize that combining global policy summaries with Q-function decomposition will significantly enhance user understanding of agent safety, compared to relying solely on local or global explanations, as shown in our evaluation section. To compute $Q_{\text{risk}}(B, a)$ and $Q_{\text{task}}(B, a)$ for an abstract state B and action a , we average $Q_{\text{risk}}(s, a)$ and $Q_{\text{task}}(s, a)$ over all concrete states $s \in B$ and all possible actions a from s :

$$Q_{\text{task}}^{\pi_{\text{task}}}(B, a) = \frac{1}{n} \mathbb{E}_{a \sim \pi(\cdot | s)} \sum_{i=1}^n Q_{\text{task}}^{\pi_{\text{task}}}(s, a) \quad (4)$$

$$Q_{\text{risk}}^{\pi_{\text{task}}}(B, a) = \frac{1}{n} \mathbb{E}_{a \sim \pi(\cdot | s)} \sum_{i=1}^n Q_{\text{risk}}^{\pi_{\text{task}}}(s, a), \quad (5)$$

where n represents the total number of concrete states within the abstract state B . These values are attached to the abstract state B in the directed graph, showing how the agent's policy π evaluates task satisfaction (4) and safety constraint violations (5) when taking action a from state B .

4.2 Formal verification via XRL abstraction

Simulation-based falsification methods such as fuzzing and search-based testing [38, 39] are effective at revealing policy failures. However, they offer no formal guarantees and can be computationally expensive. In contrast, probabilistic model checking (e.g., Storm [26, 40]) offers rigorous analysis using logical specifications. However, it faces scalability and fidelity loss resulted from abstraction. Thus, to balance tractability with formal rigor, our framework begins with model checking over an local-global presentation graph. Each node is labeled safe or unsafe by thresholding $\hat{Q}_{\text{risk}}$. We then use Storm to verify PCTL specifications of the form:

$$\varphi \equiv \mathbb{P}_{\geq p}[\neg \text{unsafe}, \mathcal{U}, \text{goal}], \quad (6)$$

asserting that with probability at least p , the goal is reached without visiting unsafe states. If safety specification φ is violated, Storm returns a counterexample that identifies a sequence of abstract states leading to any unsafe behavior.

Definition 1 (φ -Counterexample) Let $\mathcal{G}_\pi = (V, E)$ and φ be the specification above. A counterexample trace is a path $\tau = (v_0, \dots, v_T)$ such that: (1) some $v_t \in \text{unsafe}$, and (2) the probability of satisfying φ along τ is less than p .

Storm returns such a counterexample trace in $\mathcal{G}_\pi$, annotated with predicates and risk values. This interpretable trace helps developers visualize, diagnose, and repair unsafe behavior (Fig. 2).

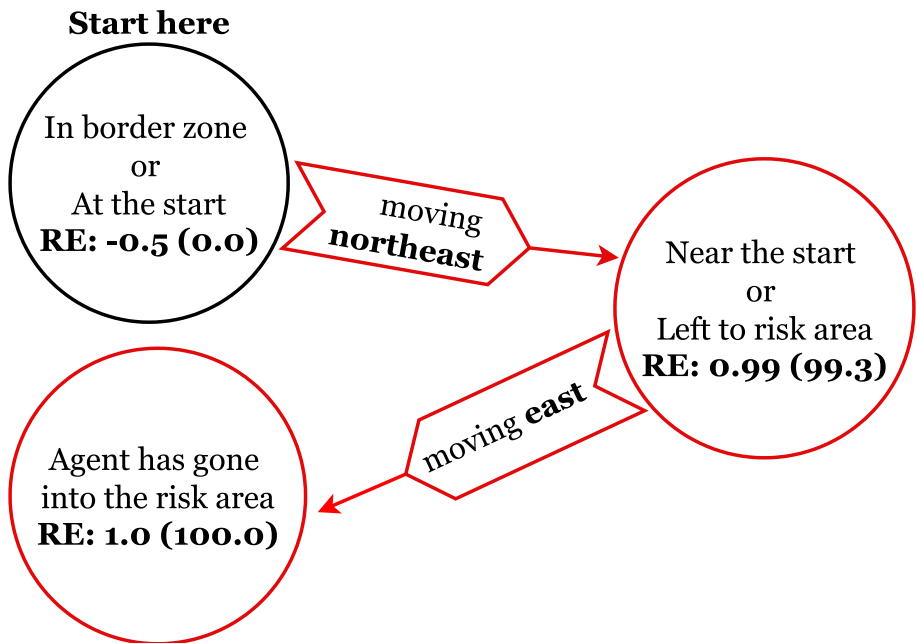


Fig. 2 A counterexample detected by Storm then visualized in CAPS in Navigation2 environment highlighted in red. Each node shows a semantic summary, associated risk estimate (RE), and its normalized value in parentheses to aid end-user interpretability

Interpretation and scope of counterexamples Counterexamples returned on $\mathcal{G}_\pi$ should be interpreted as *witnesses of unsafe behavior in the abstracted dynamics*. Under the standard assumption that (i) each abstract edge corresponds to at least one realizable transition between concrete states in the associated clusters, and (ii) the unsafe labeling rule is conservative (i.e., an abstract node labeled unsafe contains at least one concrete unsafe configuration), a counterexample trace identifies a concrete mode of failure that is realizable in the underlying system. When these assumptions are violated due to abstraction error or limited data coverage, counterexamples remain useful debugging signals, but completeness cannot be guaranteed motivating our additional testing stage.

4.3 Risk- and uncertainty-guided falsification

When formal verification yields no counterexamples, we cannot conclude with certainty that the policy satisfies the safety property in (6). This limitation stems from coverage gaps in the offline dataset, which may fail to include trajectories that expose critical failure modes. To address this, we leverage epistemic uncertainty as a proxy for data coverage: regions of the state space with high uncertainty are assumed to be underexplored and potentially risky. We introduce a falsification strategy guided by this uncertainty, combined with risk estimation, to actively search for safety violations that formal verification may have missed. This procedure is activated only when model checking reports no violation and operates under a constrained rollout budget defined by the number of seed states N and a maximum trajectory depth L .

Algorithm 1 Risk- and Uncertainty-Guided Falsification.

Input: N (risky states), L (depth limit), δ (uncertainty threshold), e (environment), α (risk threshold), π (policy), Q_{risk} (risk critic)
Output: violatedStates

```

1: model  $\leftarrow$  UncertaintyModel(numModels = 5)
2: violatedStates  $\leftarrow$  []
3: for state  $s = 1, \dots, N$  in risky states do
4:   currentState  $\leftarrow$  s
5:   done  $\leftarrow$  False
6:   violated  $\leftarrow$  False
7:   while not done and not violated do
8:      $a \leftarrow \pi(\text{currentState})$ 
9:     uncertainty  $\leftarrow$  model( $a$ , currentState) (7)
10:    for depth  $l = 1, \dots, L$  do
11:      if uncertainty  $\leq \delta$  then
12:        currentState  $\leftarrow$  mutate(currentState) (8)
13:         $a \leftarrow \pi(\text{currentState})$ 
14:        uncertainty  $\leftarrow$  model( $a$ , currentState)
15:      else
16:        break
17:      end if
18:    end for
19:    risk  $\leftarrow Q_{\text{risk}}(\text{currentState}, a)$  (5)
20:    if risk  $> \alpha$  then
21:      violated  $\leftarrow$  True
22:      violatedStates.append(currentState)
23:    end if
24:    currentState, reward, done  $\leftarrow$  e.step( $a$ )
25:  end while
26: end for

```

Seed Selection and Rollout (Alg. 1, lines 3-9) Our falsification procedure begins with the selection of high-risk initial states. Specifically, we rank each candidate state $s \in \mathcal{D}$ using the risk estimate from the model checker and select the top- N states with the highest risk score. For each seed s_0 , we roll out trajectories using the agent policy π , evaluating episodic uncertainty for each state-action pair using an ensemble of K encoders $\mathcal{E}_k$ trained via contrastive learning [37]. Each encoder computes a joint embedding:

$$\mathcal{E}_k(s, a) = \mathcal{E}_k^s(s) + \mathcal{E}_k^a(a),$$

and uncertainty is quantified as:

$$\text{Uncertainty}(s, a) = \max_{i,j} \|\mathcal{E}_i(s, a) - \mathcal{E}_j(s, a)\|_2^2. \quad (7)$$

Uncertainty Filtering and State Mutation (Alg. 1, lines 10-18) If the resulting uncertainty falls below a threshold δ , the state is assumed well-represented in $\mathcal{D}$ and unlikely to reveal novel violations. We then apply a gradient-based mutation to push toward underexplored, risky regions:

$$s'_t = s_t + \alpha \cdot \nabla_{s_t} |Q_{\text{target}}(s_{t+1}) - Q(s_t, a_t)|^2, \quad (8)$$

where $Q_{\text{target}}(s_{t+1}) = r + \gamma \max_{a'} Q(s_{t+1}, a')$. This mutation process is guided by the observation that high TD-error indicates a significant mismatch between the predicted Q value and the actual return—often signaling that the state is underexplored or it lies in a region where the agent’s learned value function generalizes poorly [41]. Thus, by climbing the TD-error gradient, we generate perturbed states that are likely to be novel or poorly understood, increasing the chance of discovering risky behaviors in regions that has not yet well captured by the offline dataset $\mathcal{D}$.

Violation Detection and Termination (Alg. 1, lines 19-23) At each step of the falsification rollout, we evaluate the safety critic Q_{risk} , and if it exceeds a threshold α , we flag it as a violation. Otherwise, the rollout proceeds until either a violation is found or the depth limit L is reached. This falsification method combines risk-aware ranking, epistemic uncertainty estimation, and gradient-guided mutation to efficiently uncover safety violations that model checking may miss—without exceeding interaction constraints.

4.4 Safety shielding for policy repair

To mitigate safety violations discovered during formal verification and falsification but without retraining the original task policy like previous methods such as [41, 42], we introduce a lightweight policy repair mechanism via a modular safety shield. Inspired by shielding techniques in safe RL [3, 5, 32], our approach *dynamically* switches between the task policy and an auxiliary safe policy based on predicted risk levels. The shield monitors the agent’s actions using the previously trained risk critic $Q_{\text{risk}}(s, a)$. When the estimated risk of an action exceeds a safety threshold ϵ , the shield intervenes and overrides the task action $a_t \sim \pi_{\text{task}}(s_t)$ with an alternative action $a_t^{\text{safe}} \sim \pi_{\text{safety}}(s_t)$ from a dedicated safe policy:

$$\text{Shield}(s_t, a_t) : Q_{\text{risk}}^{\pi_{\text{task}}}(s_t, a_t) > \epsilon. \quad (9)$$

Training the safety policy To construct π_{safety} , we adopt an adversarial regularization approach. We first synthesize an adversarial policy π_{adv} that intentionally seeks high-risk regions by maximizing expected safety costs as in [32]. The safe policy is then trained to diverge from this adversary, learning to avoid high-risk actions by solving:

$$\pi_{\text{safe}} = \arg \max_{\pi} \text{KL}(\pi \parallel \pi_{\text{adv}}), \quad (10)$$

where $\text{KL}(\cdot \parallel \cdot)$ denotes the Kullback-Leibler divergence. This objective encourages the safe policy to avoid the regions favored by the adversary, thus reducing safety violations.

Visualizing shield effects in the abstraction After applying the safety shield, we regenerate the CAPS graph to reflect policy modifications. Each edge in the updated graph is annotated as either a “task decision” or a “safety decision,” depending on whether the action was chosen by π_{task} or π_{safety} , respectively. The dominant policy at each abstract node B is inferred from the majority of associated concrete actions. This visualization enables RL practitioners to assess where and how the shield intervened, providing actionable insight into the safety-performance trade-off.

Scope of runtime safety The runtime shield is a mitigation layer, not a proof of complete safety. Since Q_{risk} is learned and may be miscalibrated in out-of-distribution regions, and since π_{safety} is also learned, we do not claim zero-risk deployment. Instead, the shield provides defense-in-depth by reducing the frequency of unsafe outcomes when the risk critic is reliable and the fallback policy can recover. In real-world deployments requiring strong guarantees, the same switching logic can be paired with a certified fallback controller and conservative monitoring to enforce hard safety envelopes, while the learned policy focuses on performance within those envelopes (Fig. 3).

5 Experiments

In this section, we present a two-fold evaluation of our framework. First, we assess user comprehension and trust in the local and global explanations generated for both safe and unsafe policies under adversarial attacks. This includes evaluating explanation fidelity and conducting user studies to understand how well users grasp and trust these explanations. Second, we evaluate the utility of these explanations in the context of formal verification by using them as inputs to model checking. We also assess the effectiveness of our falsification method in identifying counterexamples that were missed by the model checker, highlighting its value in uncovering hidden safety violations.

Tasks We evaluate our framework on three safety-critical RL domains: (1) *Navigation2* and (2) *Maze* from the CMDP Mujoco benchmark [3], and (3) a simulated *Type 1 Diabetes* insulin dosing task. *Navigation2* and *Maze* are continuous state-action environments in which the agent must reach a designated goal while avoiding obstacles, as illustrated in Fig. 4. In *Navigation2*, the agent starts at coordinates $(-50, 0)$ and must reach the goal

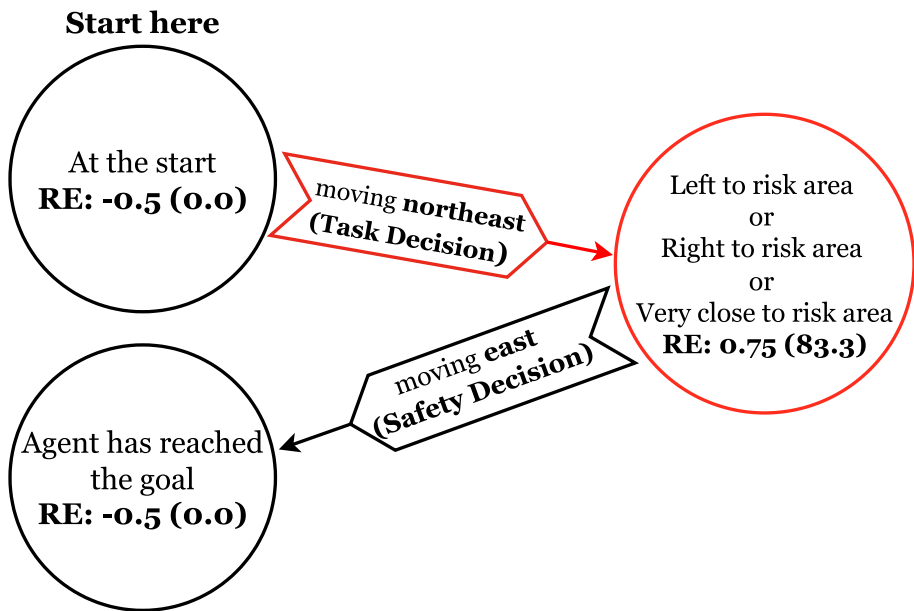
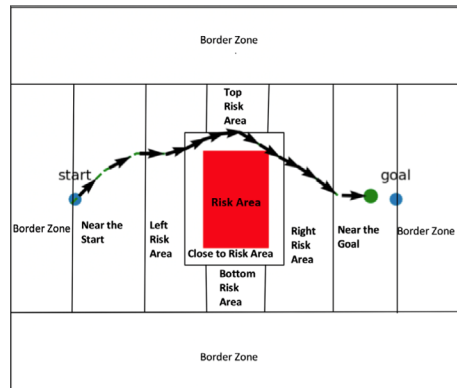


Fig. 3 CAPS graph showing shielded policy execution in Navigation2. Edges are labeled by the deciding policy—task or safety—and red highlights high-risk nodes where the shield intervened. Risk estimates (RE) are shown with normalized values in parentheses



(a) Maze Environment



(b) Navigation2 Environment

Fig. 4 Visualization of the geometric navigation environments used in our evaluation

at (0, 0) while avoiding a rectangular obstacle with its lower-left corner at (−30, −20) and upper-right corner at (−7.5, 7.5). In Maze, the agent's initial x -coordinate is uniformly sampled between −0.22 and −0.13, its y -coordinate is uniformly sampled between −0.22 and 0.22, and the goal is fixed at (0.25, 0). The agent must navigate to the goal while avoiding four rectangular obstacles: (i) top-left wall with top-left corner (−0.13, −0.28) and bottom-right corner (−0.07, −0.11), (ii) top-right wall with top-left corner (0.07, −0.28) and

bottom-right corner (0.13, 0.05), (iii) bottom-left wall with top-left corner $(-0.13, 0.21)$ and bottom-right corner $(-0.07, 0.28)$, and (iv) bottom-right wall with top-left corner $(0.07, 0.27)$ and bottom-right corner $(0.13, 0.28)$. Both environments are chosen for their geometric complexity and stochastic dynamics, which naturally induce diverse modes of failure such as near-miss trajectories and brittle navigation behaviors. The third domain is a Type 1 Diabetes insulin dosing task, which simulates patient glucose dynamics over a 24-hour period with measurements every 10 minutes. The agent selects discrete insulin doses (0 to 10 mU in 0.5 mU increments) based on physiological signals including glucose levels, gut absorption, and meal events, with the goal of maintaining glucose in the safe range of 65–105 mg/dL.¹ Unlike the geometric tasks, where safety violations manifest as immediate collisions, the diabetes domain presents long-horizon and context-sensitive safety risks, such as hypoglycemia or hyperglycemia, arising from sequences of individually reasonable but collectively unsafe actions.

The diabetes environment simulates patient glucose dynamics over a 24-hour period with measurements every 10 minutes. The agent selects discrete insulin doses (0 to 10 mU in 0.5 mU increments) based on the patient's physiological state, including current and past glucose levels, gut absorption, and meal events. The goal is to maintain glucose within the safe range of 65–105 mg/dL.² This medical task provides a partially observable and high-stakes setting, where safety violations are context-sensitive and may emerge from subtle interactions between physiological variables and agent actions.

5.1 Evaluation of local-global explanation graph

Baselines As discussed in Section 4.1, explanation methods can be broadly classified into (1) *local explanations* and (2) *global explanations*. We compare against two representative baselines: (i) our local explanation method, which presents Q_{task} and Q_{risk} values at specific time steps, and (ii) our previous, global explanation CAPS [13], which constructs a directed graph summarizing the agent's overall policy behavior. Figure 5 visualizes all three explanation methods—our local approach, global (CAPS), and our local-global explanation graph applied to a safe agent in Navigation2 trained using AdvExRL [32].

Evaluation methods We evaluate the trustworthiness of the local-global explanations using two methods: fidelity scores and user studies.

5.1.1 Fidelity

Fidelity quantifies how accurately the generated explanation reflects the true behavior of the RL agent. Higher fidelity indicates stronger alignment between the explanation graph and the agent's behavior, resulting in more reliable explanations. We evaluate fidelity across four components captured by our explanations: (i) *action*, (ii) policy selection, (iii) Q_{risk} , and (iv) Q_{task} . *Action* fidelity is computed as the proportion of actions taken by the

¹We used the simulator from <https://github.com/strongio/dosing-rl-gym> to train our agent using the SAC algorithm.

²We used the simulator from <https://github.com/strongio/dosing-rl-gym> to train our agent using the SAC algorithm

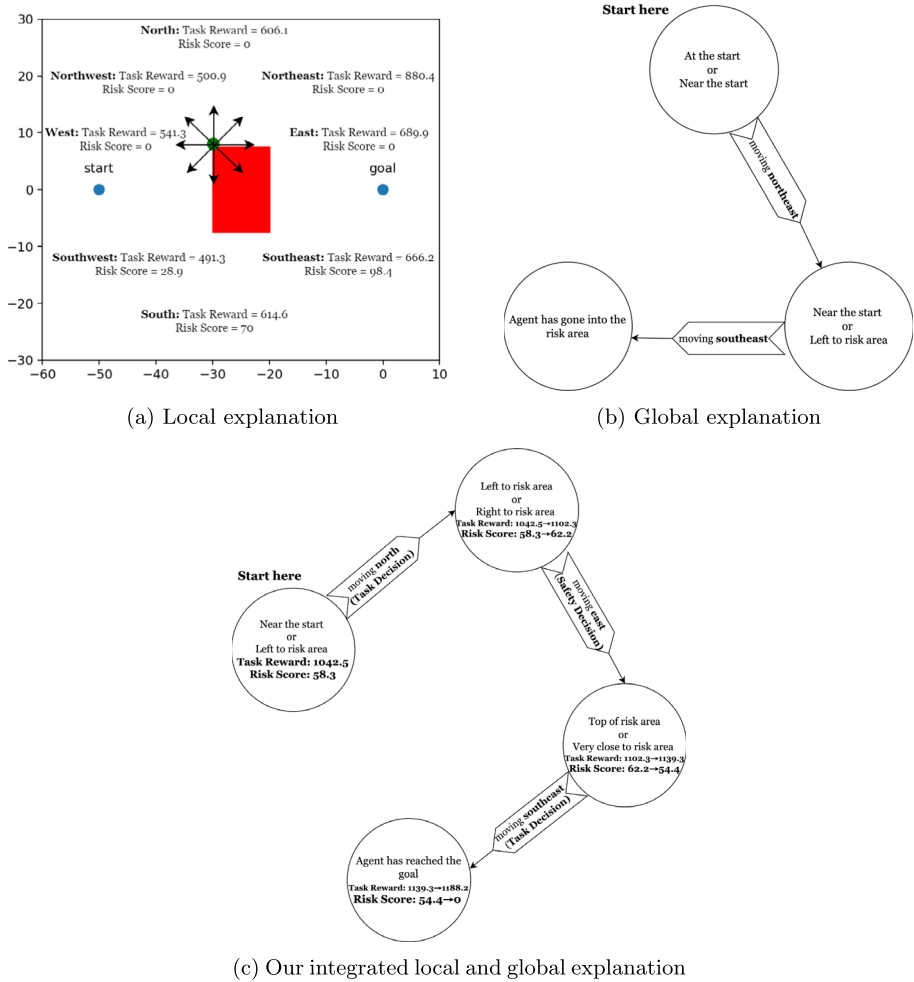


Fig. 5 Examples of generated explanations for Navigation2 task using our local explanation, global explanation from CAPS [13], and our integrated local and global explanations

RL agent that match the actions predicted by the explanation graph [43]. Policy selection fidelity measures the accuracy with which the graph identifies the policy the agent uses for decision-making. For Q_{risk} and Q_{task} , fidelity reflects how closely the graph's local explanations approximate the corresponding values produced by the agent's policy. We quantify these using the weighted average of the Normalized Root Mean Squared Error (NRMSE) between our method's estimates and the policy's values, where lower NRMSE indicates higher fidelity.

To compute all the fidelity scores, we simulate around 2,000 timesteps of agent-environment interactions for several agents: SAC [44] without attacks, SAC under a 50% attack, a safe agent trained with the safe RL algorithm AdvExRL [32], a safe agent trained with RRL-MF [3], and the SQRL [45]-trained agent. We generated and averaged fidelity scores across five graphs for each agent.

Table 1 Fidelity scores for the local-global explanations generated for different agents: SAC [44] (with and without attacks), and agents trained with AdvExRL [32], RRL-MF [3], and SQRL [45]. A higher action and policy selection ratio indicates better explanation (policy selection is only applicable for AdvExRL [32] and RRL-MF [3]). For Q_{risk} and Q_{task} , a lower NRMSE indicates better fidelity

	Action	Policy Selection	Q_{risk}	Q_{task}
SAC (w/o attack)	63.4%	-	44.5%	30.4%
SAC (w/ attack)	42.8%	-	43.1%	39.2%
advExRL	34.25%	59.2%	33.2%	32.6%
RRL-MF	31.7%	94.7%	48.4%	27.4%
SQRL	32.4%	-	62.8%	25.6%

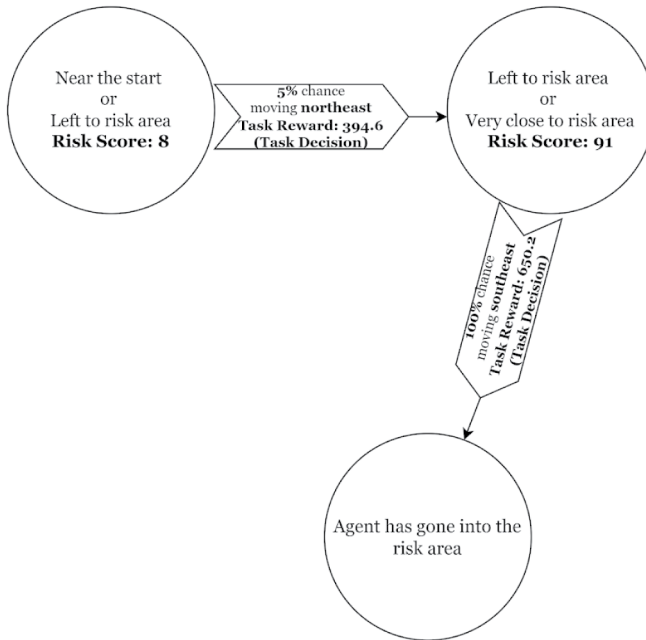
Results and discussion Table 1 summarizes the results and highlights several important trends. The SAC agent without attack achieves the highest action ratio fidelity (63.4%), demonstrating that our explanations effectively capture the agent's behavior in a stable environment. However, under attack, the action ratio fidelity for SAC drops to 42.8%, reflecting the agent's reduced consistency under adversarial conditions, which also impacts the accuracy of the explanations. Among the safe agents, AdvExRL performs notably well with a relatively high policy selection ratio (59.2%) and the lowest Q_{risk} NRMSE (33.2%), indicating that AdvExRL provides the most accurate safety explanations. This can be attributed to AdvExRL's training, which maximizes the KL divergence from an optimal adversarial policy [32], making its safety policy more optimal and, therefore, more explainable. RRL-MF achieves the highest policy selection fidelity (94.7%) due to its lower risk estimation, where most actions were taken by the task policy, as reflected in its graphs (see Fig. 6). Confirming that, RRL-MF's Q_{risk} NRMSE is relatively high (48.4%), indicating challenges in estimating risk. This is likely because RRL-MF relies on expert demonstrations for training the safe policy [3], which may not cover all possible risky scenarios, leading to less accurate safety explanations. On the other hand, SQRL excels in task-related explanations with the lowest Q_{task} NRMSE (25.6%), but it has the highest Q_{risk} NRMSE (62.8%). This discrepancy in risk estimation is likely due to SQRL's joint optimization of both task and safety requirements [45], making it harder to isolate and accurately estimate risk values.

Overall, our integrated local and global explanation method provides comprehensive explanations, but its local safety explanations (Q_{risk}) are highly influenced by the safe algorithm used to train the agents. Thus, we recommend using separate policies for safety, as seen in AdvExRL [32], which yield more optimal and interpretable safety explanations.

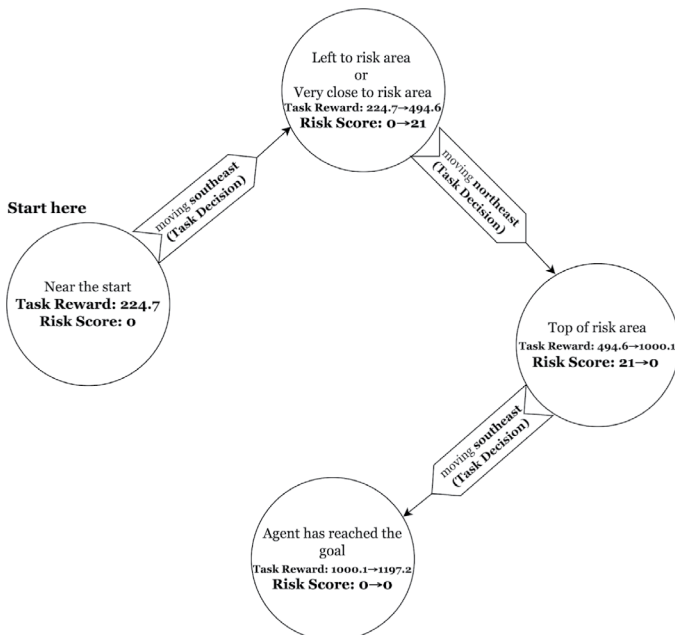
5.1.2 User studies

Overview To empirically evaluate users' trust in our local-global explanations, we conducted three user studies examining both the combined effect of local and global explanations and the individual contribution of each explanation type. Each study involved three safe agents (AdvExRL, RRL-MF, and SQRL), producing nine distinct experimental conditions. Since local explanations pertain to specific states, we selected those with the highest Q_{risk} values to present the agents' behavior in high-risk scenarios.

Start here



(a) RRL-MF without Safe Policy



(b) RRL-MF with Safe Policy

Fig. 6 Local-global explanations of RRL-MF in Navigation2 under adversarial attack

Procedure We recruited 270 participants (30 per study), including sophomores from various majors at Wake Forest University and additional participants from Prolific, with IRB approval. To ensure relevance, we applied screening filters to participant backgrounds. Each participant was randomly assigned to one study to prevent crossover effects. At the beginning, participants were introduced to the Navigation2 environment and were provided with an explanation of the safety constraints, potential adversarial attacks on the agent, Q -values, and key study terms in layperson-friendly language. They then viewed three videos of the RL agent: one showing success without attack, one showing failure under attack, and one showing success under attack when trained with a safe RL algorithm. Afterward, participants answered four questions across two scenarios: one with an agent under attack and no safety mechanism, and one with an agent trained with a safety algorithm. The questions were: (i) Q1. What action will the agent likely take in a high-risk state? (ii) Q2. Why did the agent choose that action? (iii) Q3. Will the agent succeed in safely reaching the goal? and (iv) Q4. Which agent is safer between the vulnerable and the safe agent (with a confidence rating on a 1–5 Likert scale)? (see Fig. 7). Q1 assessed participants' comprehension of explanations and the role of Q -values (Q_{task} , Q_{risk}). Q2 evaluated whether participants understood the agent's motivation—goal pursuit, risk avoidance, or response to attack—rather than the action's effectiveness. Q3 measured participants' ability to judge success in terms of task completion and safety. Q4 required participants to identify the safer agent, with only responses rated at confidence ≥ 4 reported. The Navigation2 environment was chosen because it provides sufficient complexity to highlight safety-relevant aspects of agent behavior without overwhelming participants. Participants were compensated with a base payment of \$1.50, plus a bonus of 10 cents for each correct answer.

Figure 7 displays three survey questions used in the user studies. (a) Q1 and Q2: Action prediction and motivation. Q1 asks: "Q1: Which direction did the AI player go from the location 'Near the start or Left to risk area'?" with options: North, South, East, West, Northeast, Northwest, Southeast (selected), and Southwest. Q2 asks: "Q2: Why did the AI player choose that direction from the location 'Near the start or Left to risk area'?" with options: Reaching the Goal, Avoiding the Risk Area (selected), Influenced by Attack, and I Don't Know. (b) Q3: Success prediction. Q3 asks: "Q3: Will the AI player succeed (reaching the goal)?" with options: Yes, No (selected), and I Don't Know. (c) Q4: Safety comparison with confidence rating. Q4 asks: "Survey: Overall Study Evaluation Which scenario is more effective in protecting the AI player against attacks?" with options: Scenario 1, Scenario 2, and I Don't Know.

(a) Q1–Q2: Action prediction and motivation

(c) Q4: Safety comparison with confidence rating

Fig. 7 Survey questions used in the user studies: (a) Q1 tests action prediction, (b) Q2 evaluates reasoning behind actions, and (c) Q3 measures prediction of task success

Hypotheses We will test three hypotheses:

H1: We hypothesized that using only local or global explanations would be less effective in helping users identify why the agent made certain decisions and whether or not it would succeed under attack, compared to our combined approach. In other words, participants presented with local or global explanation methods alone will struggle with Q1-Q3 for both safe and unsafe agents, while those presented with local-global explanation will perform better.

H2: We hypothesized that agents trained with separate safety policies (AdvExRL and RRL-MF) would be more interpretable than agents using joint optimization like SQRL, where the reasoning behind actions under attack is harder to pinpoint.

H3: We hypothesize that if participants had a correct mental model of the agents' strategies, they would be able to identify the safer agent.

Results for H1. Table 2 presents the results for **H1** for both unsafe (under attack agents without safety mechanisms) and safe agents (agents trained with safe RL algorithms). Results indicate that safe agents were overall more interpretable across all explanation methods, with local-global explanation providing the highest average accuracy at 77.4%. This is likely because participants could observe Q_{risk} values increasing and then decreasing after actions, which emphasized the agent's safe behavior. In contrast, for unsafe agents, where Q_{risk} remained high, participants found it more difficult to interpret the agent's reasoning, particularly with local explanations (48.14%). Notably, local-global and global explanations achieved similar performance for unsafe agents, both with an average accuracy of 60.37%. This suggests that the directed graph format used in both methods made it easier for participants to understand the agent's behavior, especially in high-risk scenarios, by providing a broader view of the agent's policy. However, the combined local-global explanation outperformed both global and local explanations for safe agents, confirming that integrating local and global explanations provides a more comprehensive understanding of the agent's behavior. Overall, our explanation method proved to be the *most effective* explanation method, offering more interpretable insights across both safe and unsafe agents to the users, thus confirming H1.

Results for H2 The results from Table 3 reveal important insights regarding the interpretability of the different safe agents (AdvExRL, RRL-MF, and SQRL) across various explanation methods (local, global, and local-global). AdvExRL consistently shows the highest interpretability across all explanation methods, particularly with local-global explanations

Table 2 Comparison of the average accuracy of participants' answers across all three safe and three unsafe agents using local, global, and local-global explanation methods

Explanation Method	Unsafe Agents	Safe Agents
Local Explanation	48.14%	68.88%
Global Explanation	60.37%	73.7%
Local-Global Explanation	60.37%	77.4%

Table 3 Comparison of the average accuracy of the participants' answers for three agents patched with AdvExRL, RRL-MF, and SQRL across all explanation methods

Explanation Method	AdvExRL	RRL-MF	SQRL
Local Explanation	74.44%	68.8%	63.3%
Global Explanation	80%	40.2%	73.3%
Local-Global Explanation	81.1%	78.9%	72.20%

Table 4 % of subjects selecting the safer agents across the nine conditions, with confidence ratings (≥ 4) reported in parentheses

	Local	Global	Local-Global
AdvExRL	40% (52.6%)	83.3% (62.9%)	70% (54.2%)
RRL-MF	93.3% (72.4%)	90% (44.8%)	90% (62%)
SQRL	76.7% (59.2%)	80% (42.9%)	83.3% (44.4%)

(81.1%). This suggests that its separate safety optimization policy enhances explainability. RRL-MF, while also using a separate safety policy, has slightly lower interpretability (78.9% with local-global), likely due to challenges in risk estimation. SQRL, which employs joint optimization of task and safety, demonstrates the lowest interpretability (63.3% with local explanations), suggesting that the joint approach makes it harder for participants to grasp the agent's decision-making process, particularly in risk-laden scenarios. This trend reinforces the value of having the safety mechanism separated, as demonstrated by AdvExRL's superior performance across all methods, particularly under attack scenarios, where its safety mechanism is more easily understood by participants. Moreover, global and local-global explanations overall provided a clearer understanding, specifically for safe agents.

Results for H3 Our results show that both local-global and global explanations achieved comparable results across all patching methods (Table 4). This is likely due to these methods' ability to provide a comprehensive view of the agent's behavior from the start state to the goal state. This allowed participants to observe the final outcomes (success or failure), which directly influenced their choice of the safer agent. Interestingly, despite the results of the objective questions in Table 3 showing that AdvExRL agents were more interpretable, participants expressed higher confidence when evaluating RRL-MF agents in the subjective questions. Overall, participants were able to recognize the safer agents with high confidence across all explanation methods, with confidence levels exceeding 50% in most cases (except for the local explanation of AdvExRL, which was lower). These results reinforce the importance of providing clear and comprehensive safety explanations to aid in understanding behavior of RL agents, particularly in safety-critical environments in which local-global explanation proved to be effective.

5.2 Formal verification, falsification, and policy repair

Evaluation methods We evaluate our formal verification, falsification, and policy repair methods across three safety-critical RL domains.

Baselines We compare our falsification component with two state-of-the-art strategies: (1) *Uncertainty-Based Search* [37], which prioritizes exploration of states underrepresented in the offline dataset using epistemic uncertainty, and (2) *DRLFuzz* [41], which expands states based on estimated risk without regard for dataset coverage. These baselines represent two complementary philosophies—uncertainty-driven exploration versus risk-driven targeting—against which we benchmark the combined guidance of our proposed method.

Evaluation metrics We evaluate our methods along four key axes:

- **Violation Count (Effectiveness):** The total number of distinct safety violations identified by Storm (model checking) and guided falsification. This measures the overall effectiveness of each method in surfacing unsafe behaviors.

- **First Violation Detection (Efficiency):** The number of rollout episodes required to detect the first safety violation during falsification. This reflects how quickly each method can surface a critical failure under a constrained rollout budget.
- **Execution Time (Efficiency):** The total wall-clock time required to perform Storm verification and guided falsification, reported separately. This complements episode-based efficiency by capturing the computational cost of each component. All experiments were conducted on a machine with an AMD EPYC Milan CPU (x86_64 architecture), 8 logical CPUs, 1 physical core per CPU, and 21.6 GB of RAM.
- **Violation Diversity:** The mean pairwise distance among latent representations of unsafe trajectories discovered via falsification. Higher values mean greater spatial and behavioral spread in the state-action space, reflecting broader coverage of potential failure modes.
- **Violation Novelty:** The average distance between discovered unsafe trajectories and those in the offline dataset, measured in the same latent space and computed via the average Mahalanobis distance. This quantifies the extent to which the detected failures represent novel behaviors not captured during training.

For evaluation, we pose several research questions (RQs) below (the tabular results are included in Appendix B):

- **RQ1:** How effective is the proposed framework in detecting safety violations compared to standalone model checking, our falsification strategy, and baseline methods? And How novel are the violations relative to the trajectories in the offline dataset?
- **RQ2:** How diverse are the violations discovered by our guided falsification strategy?
- **RQ3:** How efficient is the falsification strategy in terms of both the number of trajectories required to detect the first violation and the overall falsification runtime?
- **RQ4:** What is the impact of using CAPS for abstraction on formal verification performance compared to a clustering-based abstraction using KMeans?
- **RQ5:** How does runtime safety shielding affect the frequency of safety violations under the same verification-falsification framework?

Results and discussion We apply our falsification strategy even when Storm detects violations to assess overlap and complementarity between verification and search, reporting aggregate results across environments and baselines. As part of an ablation, we vary the mutation depth (state mutation frequency) using values of 1, 10, and 20. Results in Navigation2 and Maze remain consistent, suggesting limited sensitivity to depth due to immediate safety feedback. In contrast, Dosing benefits from deeper mutations, consistent with the domain's delayed physiological responses, where safety violations such as hypoglycemia emerge only after a series of cumulative missteps rather than immediate feedback. We also rerun experiments with a safety-shielded agent to evaluate the effectiveness of runtime policy repair within our framework.

RQ1 (Fig. 8) This question examines how effective each method is at uncovering safety violations and how different the discovered violations are from those encountered during training. Across all environments, the Storm model checker identified a small number of violations (5–11 trajectories). These are formally verified counterexamples, but their

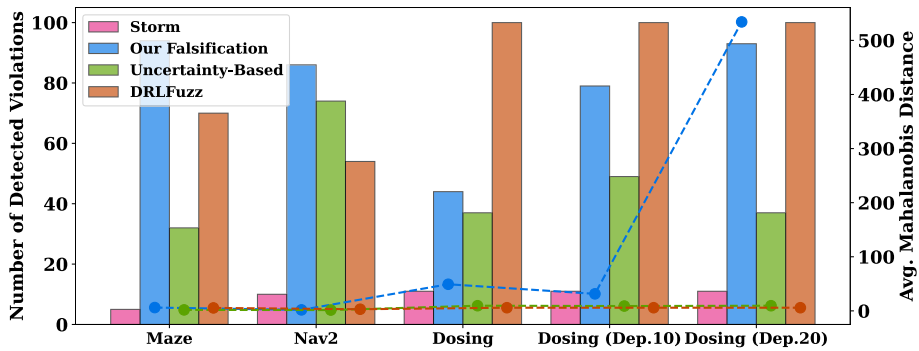


Fig. 8 RQ1: Number of detected violations per method across environments, with Mahalanobis distance showing how far the violations are from the dataset. Our framework (Storm + Falsification) consistently finds more violations with higher novelty

coverage is inherently limited by the fidelity of the abstraction and the coverage of the offline dataset. While our framework terminates after model checking if a violation is found, we executed the falsification stage regardless to assess its added benefit. Our guided falsification approach outperformed both DRLFuzz and uncertainty-based search in Maze and Navigation2, finding 94 and 86 violations, respectively. In the Dosing environment, DRLFuzz achieved the highest violation count (100), though this was accompanied by relatively lower novelty. Specifically, the average Mahalanobis distance of DRLFuzz violations in Dosing (5.8) was far lower than that of our approach, which exceeded 49 in standard depth and reached 534 at depth 20. This distance metric indicates how far the violations are from the distribution of the training data—suggesting that our method detects failures in less explored, higher-risk regions of the state space. In contrast, DRLFuzz, though effective in raw count, tends to revisit more common failure regions. This distinction is especially critical in complex, temporally sensitive domains like insulin dosing, where new failure modes often emerge in rare or sparsely sampled conditions.

RQ2 (Fig. 9) To evaluate the diversity of the detected violations, we measure the mean pairwise distance between the violating trajectories discovered by each method. This metric captures how semantically varied the unsafe behaviors are within each method's outputs, offering insight into whether the detected violations are close to each other or concentrated in the same regions of the state space. The results in Fig. 9 are plotted on a logarithmic scale to emphasize differences across environments, especially where absolute values are small. In the Dosing environments (standard, depth 10, and depth 20), our guided falsification strategy consistently achieves higher diversity than DRLFuzz and significantly outperforms Uncertainty-Based search in two out of three settings. For example, in the base Dosing task, our method reaches a mean pairwise distance of 71.54, compared to 27.82 for Uncertainty-Based and 34.96 for DRLFuzz. These results demonstrate that our method explores a broader range of unsafe behaviors in complex, real-world environments like medical dosing. However, in simpler domains like Navigation2 and Maze, DRLFuzz occasionally achieves higher diversity than our method. For instance, in Navigation2, DRLFuzz yields a pairwise distance of 11.36 versus 5.23 for our method, and in Maze, its diversity is slightly higher (0.30 vs. 0.15). This suggests that while our risk-guided strategy excels in high-

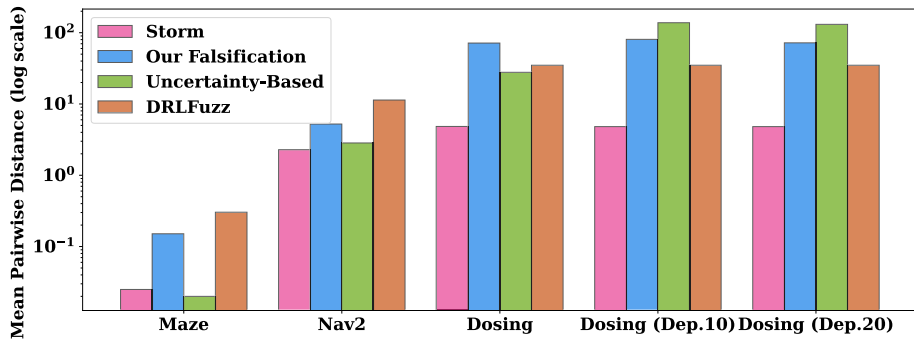
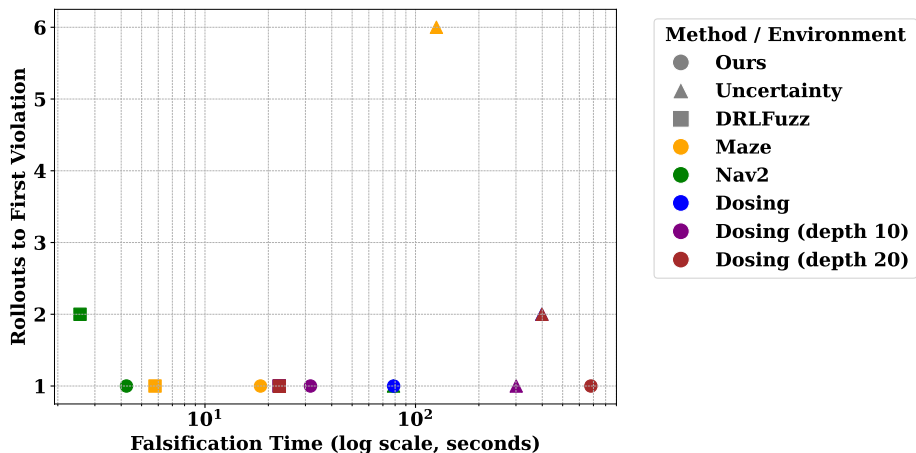


Fig. 9 RQ2: Mean pairwise distance of violating trajectories discovered by each falsification method across environments. Higher values indicate more diverse safety violations. Values are plotted on a logarithmic scale to emphasize differences in small-scale environments like Maze and Navigation2

dimensional, structured domains, it may be less effective in simpler environments where some randomness in the search—like that used by DRLFuzz—can help uncover more varied violations. Overall, these results indicate that our approach reliably captures a wide range of unsafe behaviors in complex domains, which is especially important for applications like healthcare. The log-scaled plot highlights that even modest differences in small environments can reflect distinct behavioral patterns, reinforcing the value of evaluating diversity alongside violation count.

RQ3 (Fig. 10) We assess efficiency using two metrics: (1) the number of trajectories required to discover the first safety violation, and (2) the total time taken to perform falsification. As shown in Fig. 10, our falsification strategy consistently identifies the first violation with minimal rollout cost—requiring only a single trajectory across all environments, including the more complex Dosing tasks. In contrast, the uncertainty-based strategy takes longer in both metrics, especially in Maze and the deeper Dosing scenarios (6 and 2 trajectories



respectively), with falsification times reaching up to 398 seconds. DRLFuzz shows strong performance in rollout budget and time, often matching our method in the number of rollouts to the first violation (1 or 2 trajectories) and achieving the lowest execution time overall (e.g., only 2.5s in Navigation2 and 22s in Dosing). However, as shown in RQ1 and RQ2, this comes at the cost of reduced trajectory diversity (in Dosing) and less effective exploration of underrepresented or high-risk regions. In summary, our falsification approach strikes a favorable balance: it matches DRLFuzz in minimal trajectory requirements, is significantly faster than uncertainty-based methods in all domains except Dosing (depth 20), and—when paired with its stronger violation count and competitive diversity from earlier RQs—demonstrates a strong trade-off between speed, effectiveness, and exploration completeness.

RQ4 To assess the impact of abstraction choice on verification performance, we replaced CAPS with a KMeans-based policy abstraction and compared results on violation count and action fidelity across Maze, Navigation2, and Dosing. Although both methods generated comparable numbers of abstract states, CAPS enabled Storm to detect 5–11 violations per environment, while KMeans found only 2 violations in Navigation2 and none in Maze or Dosing. This discrepancy stems from the difference in action fidelity: CAPS preserved the task policy’s action in over 83% of abstract states in Navigation2, while KMeans achieved only 60%. Similar trends appear in Maze (51% vs. 36%) and Dosing (37% vs. 25%). Despite the simplicity of KMeans, its poor alignment with the agent’s actual decision-making led to abstractions that do not accurately capture policy behavior. As a result, formal safety analysis using KMeans-based graphs becomes unreliable. These findings underscore that CAPS not only enhances interpretability but also maintains fidelity crucial for trustworthy verification.

RQ5 (Fig. 11) We report the number of violations detected by Storm and by our falsification strategy with and without shielding in the Maze and Navigation2 environments. In both environments, Storm fails to detect any violations once the shield is active, despite the presence of remaining violations found via falsification. This shows that shielding may help the policy

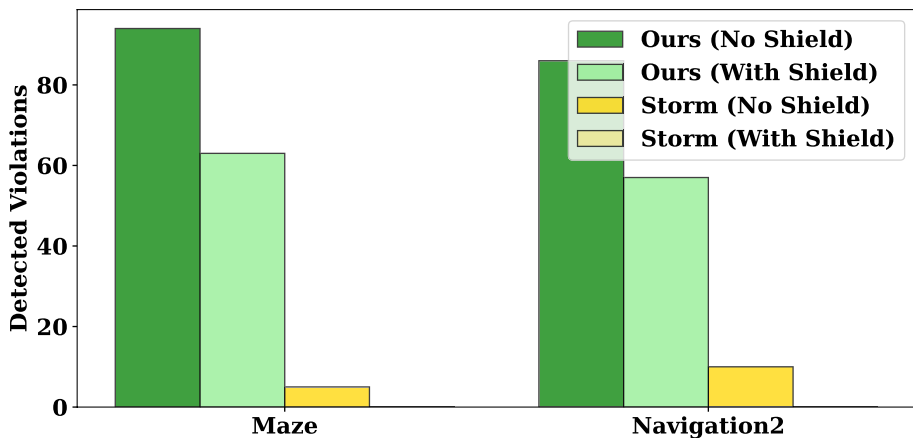


Fig. 11 RQ5: Impact of safety shielding on detected violations in Maze and Navigation2. After shielding, Storm detects no violations, while falsification finds fewer failures—highlighting both the shield’s effectiveness and the continued need for falsification to ensure safety

mistakenly pass formal safety specifications, which supports our design of falsification as a necessary second line of verification when the model checker fails to flag unsafe behavior. Our falsification reveals that shielding reduces the number of detected violations by 33% in Maze (from 94 to 63) and 34% in Navigation2 (from 86 to 57). These results show that the shield effectively blocks risky policy decisions identified at runtime by the risk critic. We omit shielding results for the Dosing environment as its safety constraints are not directly encoded in the environment's dynamics or terminal conditions as in CMDP benchmarks of Maze and Navigation2. Violations in this domain (e.g., hypoglycemia) are temporally extended and depend on delayed physiological effects, which makes training an effective runtime shield non-trivial. This limitation arises from the domain design—not from our framework—and motivates future work on designing temporally-aware shields for safety-critical medical applications. While the runtime shield significantly reduces violations, it may fail to provide absolute guarantees in scenarios where the risk critic encounters out-of-distribution states or where the environment's stochasticity exceeds the fallback policy's recovery capabilities. In such cases, the framework serves as a defense-in-depth mechanism that minimizes harm and provides interpretable logs for post-failure analysis, rather than a guarantee of zero-risk deployment.

6 Conclusion

AEGIS-RL addressed the challenge of verifying and falsifying safety properties in reinforcement learning policies under limited data coverage by designing a hybrid framework that combines explainable abstraction, probabilistic model checking, and risk-aware falsification. While effective in diverse safety-critical tasks, the proposed approach has two main limitations. First, CAPS does not currently support image-based observation spaces; future work will explore extending CAPS or developing new abstractions for high-dimensional inputs. Second, this work omits shielding results for the Dosing task, as its temporally extended safety violations and partially observable dynamics make runtime repair nontrivial. This motivates future research on designing temporally-aware shields tailored to medical domains with delayed effects and implicit constraints.

Author Contributions T.L prepared Figs. 1, 8, 9, and 10 T.L prepared algorithm 1 T.L and R.S implemented the model checking and falsification methods R.S, Thai Le, and S.A prepared Figs. 2, 3, 4, 5, 6, and 7 D.G implemented state mutation method R.S and Thai L wrote the main manuscript text All authors reviewed the manuscript

Data Availability Data is provided within the manuscript.

Declarations

Competing Interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Kim, D., Moon, S., Hostallero, D.E., Kang, W.J., Lee, T., Son, K., & Yi, Y. (2020). Safe reinforcement learning: A survey. [arXiv:2005.00903](https://arxiv.org/abs/2005.00903)
- Geibel, P. (2006). Reinforcement learning for MDPs with constraints. *European Conference on Machine Learning*, pp. 646–653.
- Thananjeyan, B., Balakrishna, A., Nair, S., Luo, M., Srinivasan, K., Hwang, M., Gonzalez, J. E., Ibarz, J., Finn, C., & Goldberg, K. (2021). Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones. *IEEE Robotics and Automation Letters*, 6(3), 4915–4922.
- Bastani, O. (2021). Safe Reinforcement Learning with Formal Guarantees. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(17), 15102–15110.
- Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., & Topcu, U. (2018). Safe Reinforcement Learning via Shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).
- Mihatsch, O., & Neuneier, R. (2002). Risk-sensitive reinforcement learning. *Machine learning*, 49(2), 267–290.
- Ruggeri, F., Russo, A., Inam, R., & Johansson, K.H. (2025). Explainable reinforcement learning via temporal policy decomposition. [arXiv:2501.03902](https://arxiv.org/abs/2501.03902)
- Corsi, D., Marchesini, E., & Farinelli, A. (2021). Formal verification of neural networks for safety-critical tasks in deep reinforcement learning. In *Proceedings of the 37th Conference on Uncertainty in Artificial Intelligence*.
- Cheng, Z., Yu, J., & Xing, X. (2025). A survey on explainable deep reinforcement learning. [arXiv:2502.06869](https://arxiv.org/abs/2502.06869)
- Deshmukh, J.V., Jin, X., Kapinski, J., Ueda, K., & Butts, M. (2015). Stochastic search techniques for reachability analysis of markov decision processes. In *Tools and Algorithms for the Construction and Analysis of Systems*, pp. 211–229. Springer.
- Karunakaran, P., & Seshia, S.A. (2020). Counterexample-guided reinforcement learning with model-based exploration. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Shefin, R.S., Rahman, M.A., Le, T., & Alqahtani, S. (2025). xSRL: Safety-aware explainable reinforcement learning-safety as a product of explainability. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, pp. 1932–1940.
- McCalmon, J., Le, T., Alqahtani, S., & Lee, D. (2022). CAPS: Comprehensible abstract policy summaries for explaining reinforcement learning agents. In *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems. AAMAS '22*, pp. 889–897. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC.
- Molnar, C. (2020). Interpretable Machine Learning, 1st edn. <https://christophm.github.io/interpretable-ml-book>
- Hayes, B., & Shah, J.A. (2017). Improving robot controller transparency through autonomous policy explanation. In *Proceedings of the 2017 ACM/IEEE International Conference on Human-robot Interaction*, pp. 303–312.
- Topin, N., & Veloso, M. (2019). Generation of policy-level explanations for reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 2514–2521.
- Amir, D., & Amir, O. (2018). Highlights: Summarizing agent behavior to people. In *Proceedings of the 17th International Conference on Autonomous Agents and Multiagent Systems*, pp. 1168–1176.
- Zahavy, T., Ben-Zrihem, N., & Mannor, S. (2016). Graying the black box: Understanding DQNs. In M.F., Balcan, & K.Q. Weinberger (Eds.), *Proceedings of the 33rd International Conference on Machine Learning*. Proceedings of Machine Learning Research, vol. 48, pp. 1899–1908. PMLR, New York, New York, USA. <https://proceedings.mlr.press/v48/zahavy16.html>
- Huber, T., Limmer, B., & André, E. (2022). Benchmarking perturbation-based saliency maps for explaining atari agents. *Frontiers in Artificial Intelligence*, 5, Article 903875.
- Puri, N., Verma, S., Gupta, P., Kayastha, D., Deshmukh, S., Krishnamurthy, B., & Singh, S. (2019). Explain your move: Understanding agent actions using specific and relevant feature attribution. [arXiv:1912.12191](https://arxiv.org/abs/1912.12191)
- Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., & Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In *IJCAI/ECAI Workshop on Explainable Artificial Intelligence*.
- Kwiatkowska, M., Norman, G., & Parker, D. (2011). Prism 4.0: Verification of probabilistic real-time systems. In *International Conference on Computer Aided Verification (CAV)*, pp. 585–591. Springer, Snowbird, USA.
- Dehnert, C., Junges, S., Katoen, J.-P., & Volk, M. (2017). A storm is coming: A modern probabilistic model checker. In *International Conference on Computer Aided Verification*, pp. 592–600. Springer.
- Li, L., Walsh, T. J., & Littman, M. L. (2006). Towards a unified theory of state abstraction for MDPs. *AI&M*, 1(2), 3.

25. Baier, C., & Katoen, J.-P. (2008). *Principles of Model Checking* (Vol. 26202649). Cambridge, Massachusetts: The MIT Press.
26. Hahn, E.M., Hartmanns, A., Hensel, C., Klauck, M., Kretinsky, J., Parker, D., Quatmann, T., Ruijters, E., & Steinmetz, M. (2019). Storm: A modern probabilistic model checker. In *International Conference on Computer Aided Verification*, pp. 592–600. Springer, Cham.
27. Liu, T., McCalmon, J., Rahman, A., Lee, T., Le, D., & Alqahtani, S. (2023). A policy-graph approach to explain reinforcement learning agents: A novel policy-graph approach with natural language and counterfactual abstractions for explaining reinforcement learning agents. *Autonomous Agents and Multi-Agent Systems Journal*.
28. Liu, B., Xia, Y., Yu, & P.S. (2000). Clustering through decision tree construction. In *Proceedings of the 9th International Conference on Information and Knowledge Management. CIKM '00*, pp. 20–29. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/354756.354775>
29. Topin, N., & Veloso, M. (2019). Generation of policy-level explanations for reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, pp. 2514–2521.
30. Bekkemoen, Y., & Langseth, H. (2024). ASAP: attention-based state space abstraction for policy summarization. In *Asian Conference on Machine Learning*, pp. 137–152. PMLR.
31. Peng, S., Liu, S., Zhi, D., Wang, P., Xu, C., Chen, C., & Zhang, M. (2025). ATA: An abstract-train-abstract approach for explanation-friendly deep reinforcement learning. *Neural Networks*, 107749.
32. Rahman, M.A., Liu, T., & Alqahtani, S.M. (2023). Adversarial behavior exclusion for safe reinforcement learning. In *IJCAI*, pp. 483–491.
33. Settles, B. (2011). From theories to queries: Active learning in practice. In I. Guyon, G. Cawley, G. Dror, V. Lemaire, A. Statnikov (Eds.), *Active Learning and Experimental Design Workshop In Conjunction with AISTATS 2010*. Proceedings of Machine Learning Research, vol. 16, pp. 1–18. PMLR, Sardinia, Italy. <https://proceedings.mlr.press/v16/settles11a.html>
34. Cohn, D. A., Ghahramani, Z., & Jordan, M. I. (1996). Active learning with statistical models. *Journal of Artificial Intelligence Research*, 4, 129–145.
35. Bachman, P., Sordoni, A., & Trischler, A. (2017). Learning algorithms for active learning. In *International Conference on Machine Learning (ICML)*, pp. 301–310.
36. Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline reinforcement learning: Tutorial, review, and perspectives on open problems. [arXiv:2005.01643](https://arxiv.org/abs/2005.01643)
37. Dukkipati, A., Ayyagari, R.S., Dasgupta, B., Dutta, P., & Onteru, P.R. (2025). Active reinforcement learning strategies for offline policy improvement. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence (AAAI-25)*. AAAI Press, Philadelphia, USA.
38. Karakovsky, S., & Togelius, J. (2012). The mario ai benchmark and competitions. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 55–67.
39. Manès, V. J., Han, H., Han, C., Cha, S. K., Egele, M., Schwartz, E. J., & Woo, M. (2019). The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering*, 47(11), 2312–2331.
40. Wang, T.-W., Huang, X., Li, B., Jha, S., Jha, S., & Chen, J. (2020). Towards verification of neural-network control systems. [arXiv:2003.06139](https://arxiv.org/abs/2003.06139)
41. Wan, X., Li, T., Lin, W., Cai, Y., & Zheng, Z. (2024). Coverage-guided fuzzing for deep reinforcement learning systems. *Journal of Systems and Software*, 210, 111963. <https://doi.org/10.1016/j.jss.2024.111963>
42. Tappler, M., Pferscher, A., Aichernig, B.K., & Könighofer, B. (2024). Learning and repair of deep reinforcement learning policies from fuzz-testing data. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3623311>, <https://doi.org/10.1145/3597503.3623311>
43. Markus, A. F., Kors, J. A., & Rijnbeek, P. R. (2021). The role of explainability in creating trustworthy artificial intelligence for health care: a comprehensive survey of the terminology, design choices, and evaluation strategies. *Journal of Biomedical Informatics*, 113, 103655.
44. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, pp. 1861–1870. PMLR.
45. Srinivasan, K., Eysenbach, B., Ha, S., Tan, J., & Finn, C. (2020). Learning to be Safe: Deep RL with a Safety Critic. [arXiv:2010.14603](https://arxiv.org/abs/2010.14603)

Authors and Affiliations

Tuan Le¹ · Risal Shefin² · Debashis Gupta² · Thai Le³ · Sarra Alqahtani²

✉ Thai Le
tle@iu.edu

Tuan Le
tuanle@iastate.edu

Risal Shefin
shefrs24@wfu.edu

Debashis Gupta
guptd23@wfu.edu

Sarra Alqahtani
sarra-alqahtani@wfu.edu

¹ Computer Science, Iowa State University, Ames, Iowa, USA

² Computer Science, Wake Forest University, Winston-Salem, North Carolina, USA

³ Computer Science, Indiana University, Bloomington, Indiana, USA